



Link to my GitHub Repo



Link to this
Presentation

Customer Stories Using Omnissa Horizon's REST API

Matt Heldstab
Systems Engineer – Minnesota State
Colleges and Universities



Who am I?

- Systems Engineer for Minnesota State (26y)
- Server Virtualization since 2006
- Desktop Virtualization since 2009
- VMware vExpert x 12
- Omnissa Tech Insider / EUC Champ x11
- VMUG Leader in Minneapolis
- Former VMUG Vice President / Director
- VCP-DCV/DT/VCF Admin/VCF Architect
- Rabid vCommunity Advocate
- Blog at www.vmatt.net



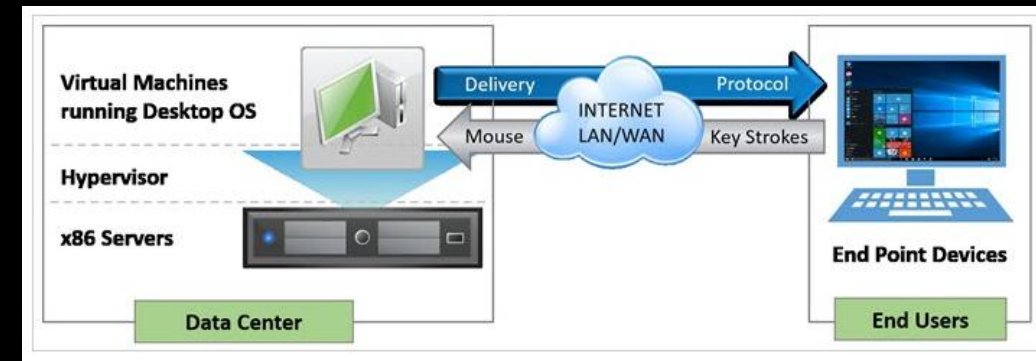
Let's chat about VDI

- What is VDI?
- VDI at the Minnesota State System Office
- Current Architecture / Strategy
- Opportunities for Automation
- Omnissa Horizon REST API
- Use-cases at Minnesota State
- Live Demo

What is VDI?

- Virtual Desktop Infrastructure

- Physical compute / storage / network infrastructure
 - A Hypervisor to create and manage virtual desktops
 - Virtual Machines that provide the OS for the user (Windows or Linux)
 - Connection Brokers / Gateways to Authenticate/Authorize users and allow access
 - Load Balancers (optional)
 - A client installed on an endpoint (optional if there is a web client)
 - Display Protocol that allows streaming to the desktop from your client device
 - Other components may include Active Directory, DHCP, KMS, File Servers, Persona Software
 - Antivirus / Antimalware
-
- VDI is typically on-premises.
 - VDI in the cloud is usually referred to as DaaS (Desktop as a Service)



VDI at the Minnesota State System Office

- Virtual Desktop Infrastructure components

- Physical compute / storage / network infrastructure – Dell MX Blades, HPE Alletra Storage, Cisco/Palo Alto
- A Hypervisor to create and manage virtual desktops – VMware vSphere 8 (VVF for VDI)
- Virtual Machines that provide the OS for the user – Windows 11
- Connection Brokers / Gateways – Omnisia Horizon Connection Servers and Unified Access Gateways
- Load Balancers – Citrix Netscaler VPX
- A client installed on an endpoint – Mix of Omnisia Horizon Client and Web Client running
- Display Protocol that allows streaming to the desktop from your client device – Allow users the choice of Blast, PCoIP or RDP, and the web client.
- Other components may include Active Directory, DHCP, KMS, File Servers, Persona Software – Mostly Microsoft products here, and Liquidware Profile Unity for persona management.
- Antivirus / Antimalware – Microsoft Defender for Endpoint

Current Architecture and Strategy

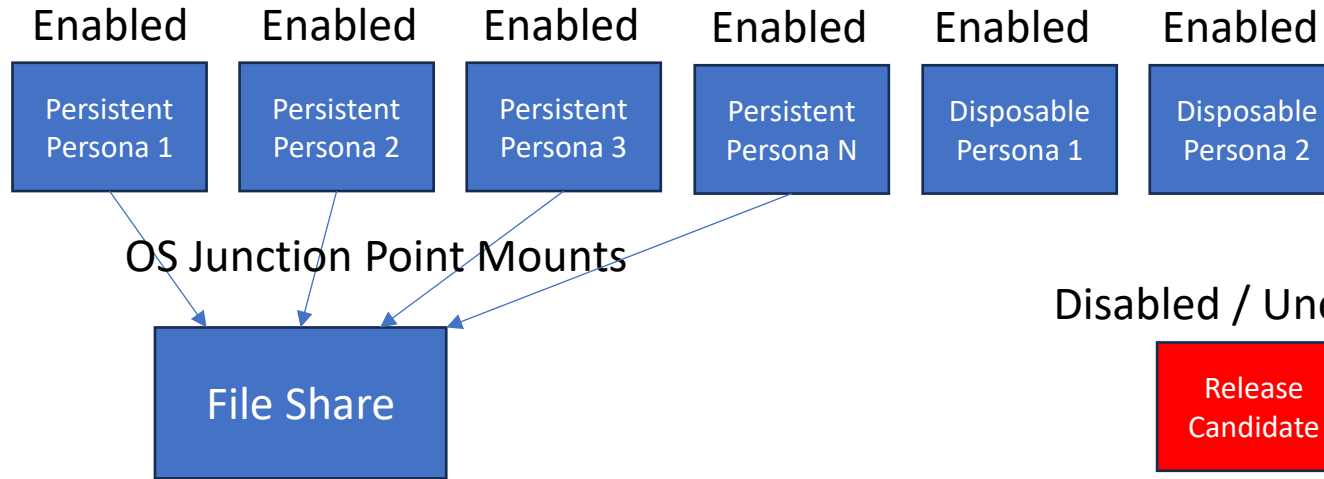
- Approximately 500 customer users
 - Range from daily, all-day use, to sporadic for temporary remote access
 - System Office and 6 of our 54 campuses
 - 80% Non-Persistent, Floating with a persistent persona
 - 20% Non-Persistent, Floating with a disposable persona
 - Redundant Gateways and Connection Servers – all load balanced with Netscaler

Current Architecture and Strategy

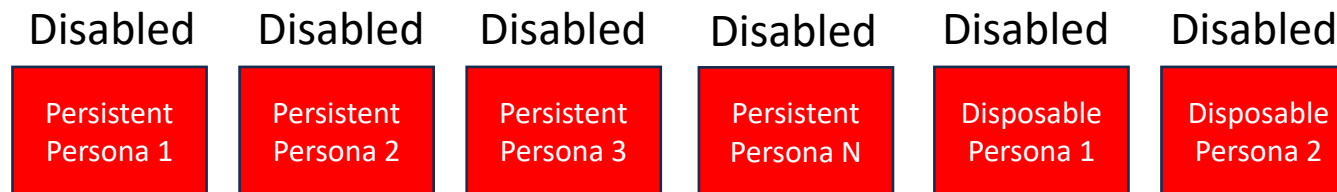
- Campuses have between 1 and 4 network security zones, each with a desktop pool
- Each pool contains a MAX limit between 20 and 70 machines with a minimum/spare machines set to 10% of the MAX
- Each pool deployment has 2 pools (A side and B side)
- Moves from A to B to A after each in-band or out-of-band

Current Architecture and Strategy (Operations)

A Pools



B Pools



Enabled
Unpatched

Disabled
Unpatched

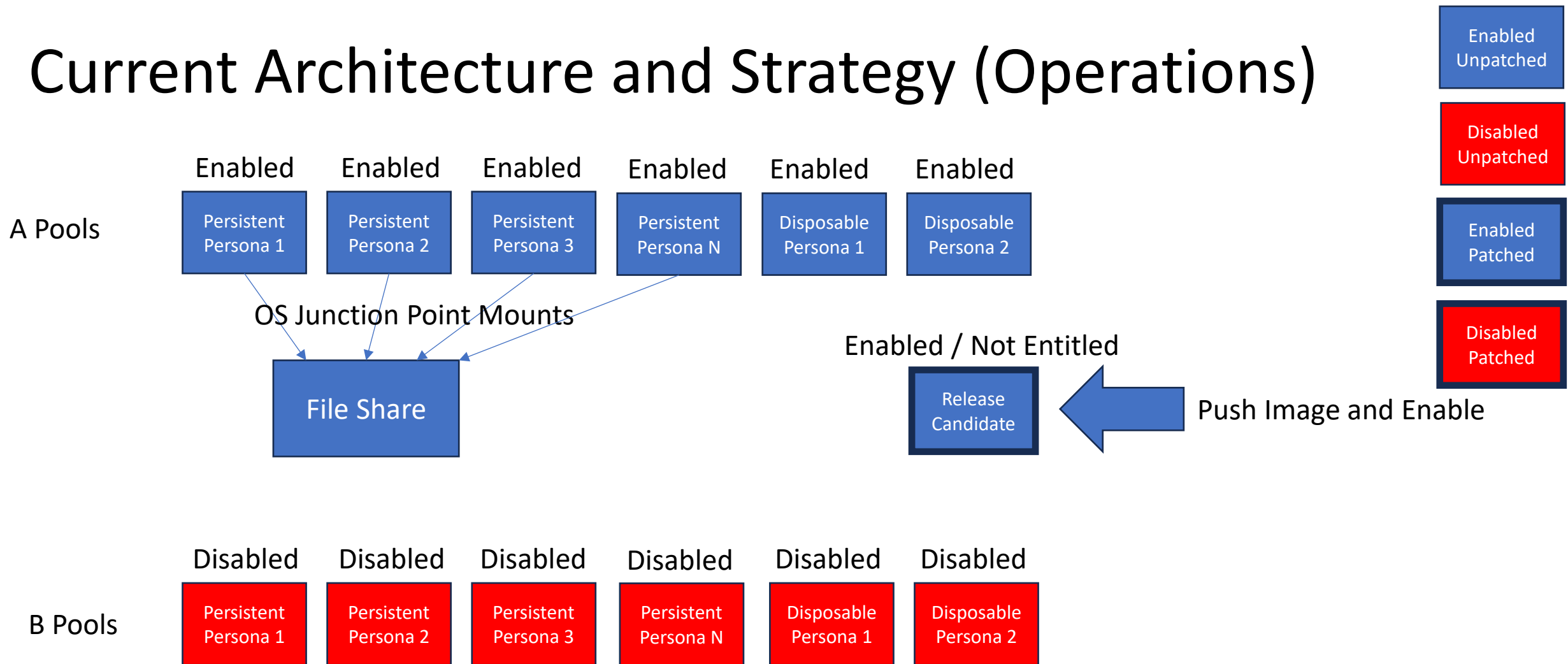
Enabled
Patched

Disabled
Patched

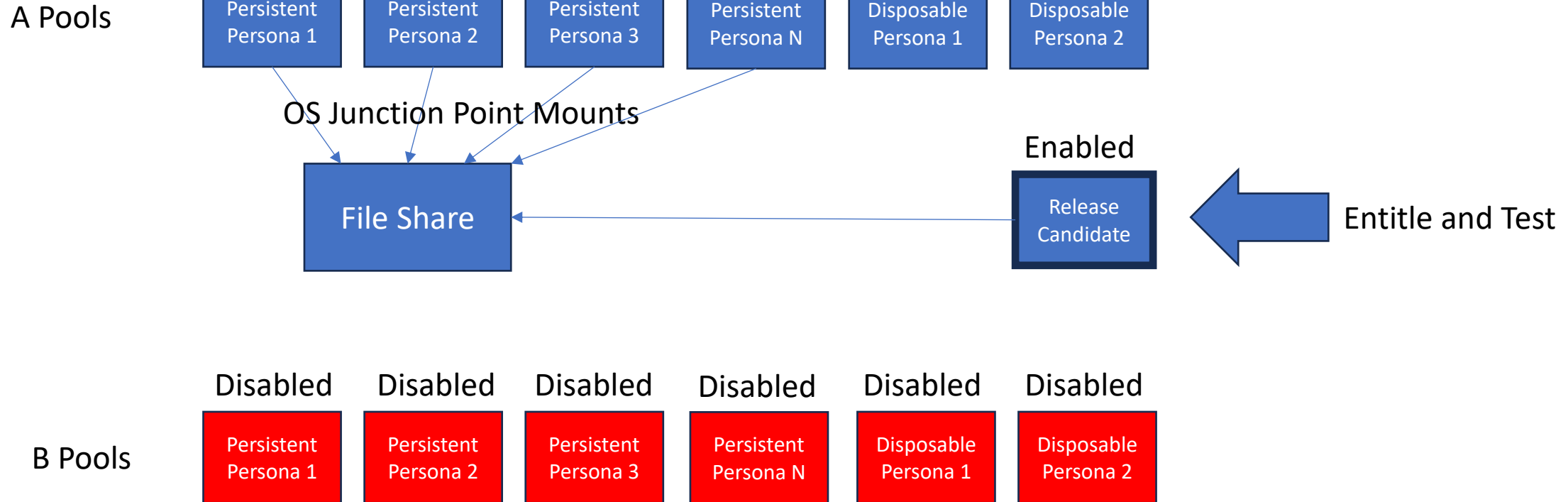
Disabled / Unentitled

Release
Candidate

Current Architecture and Strategy (Operations)



Current Architecture and Strategy (Operations)



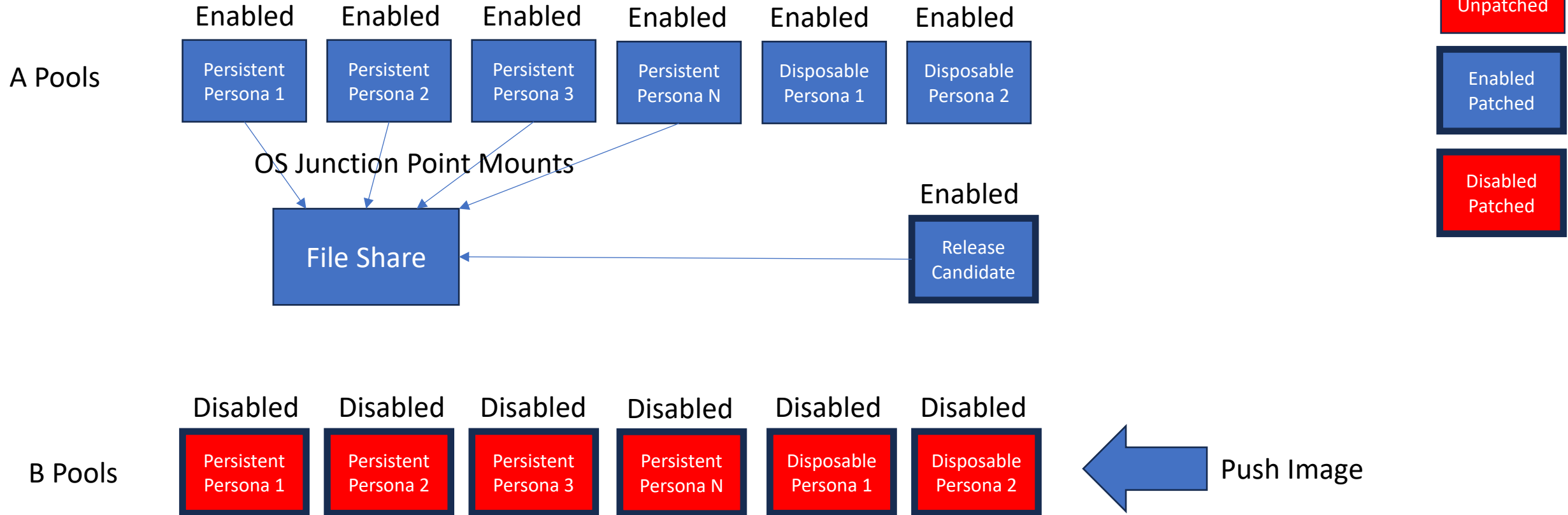
Enabled
Unpatched

Disabled
Unpatched

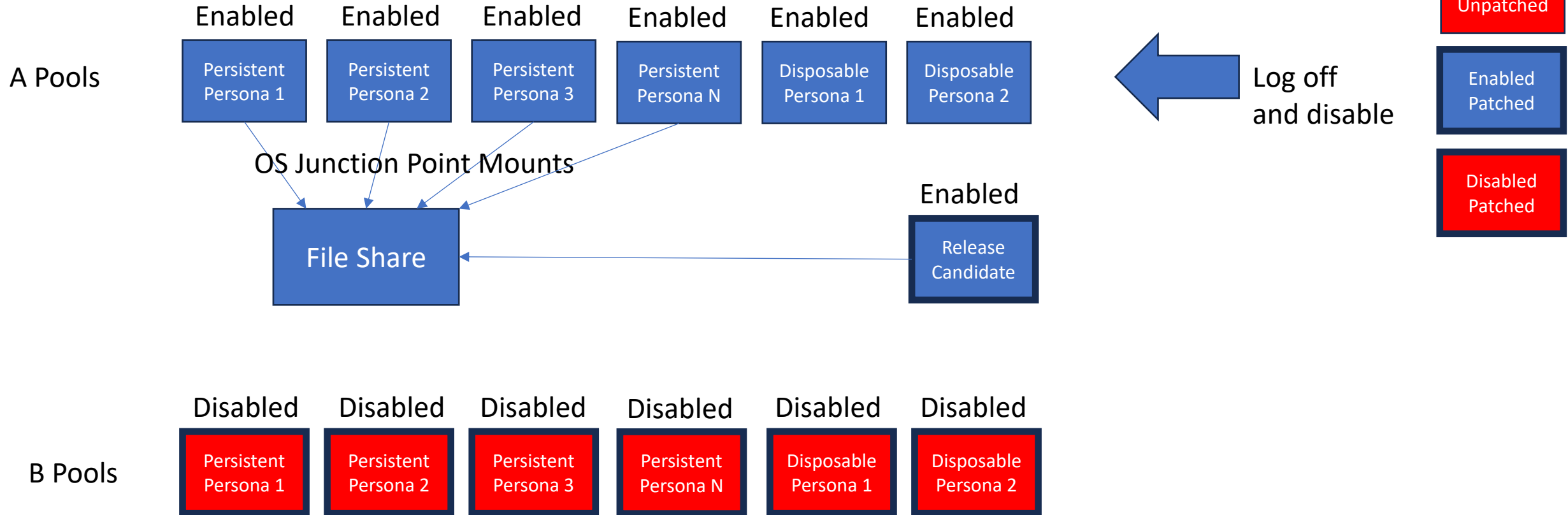
Enabled
Patched

Disabled
Patched

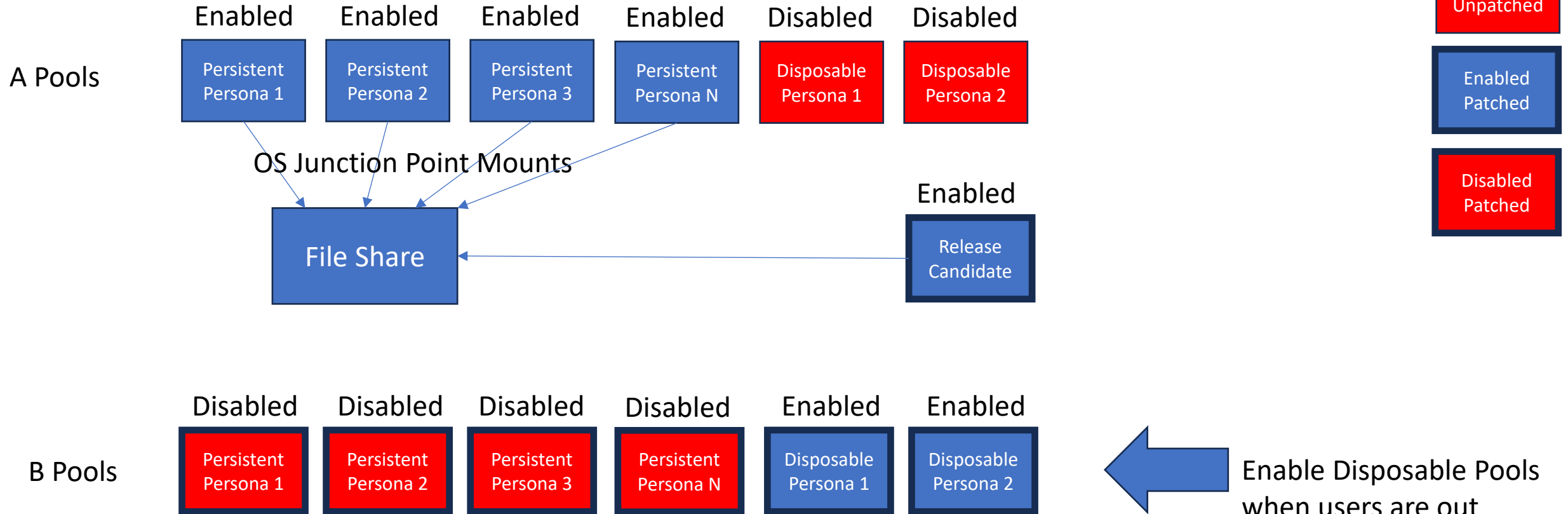
Current Architecture and Strategy (Operations)



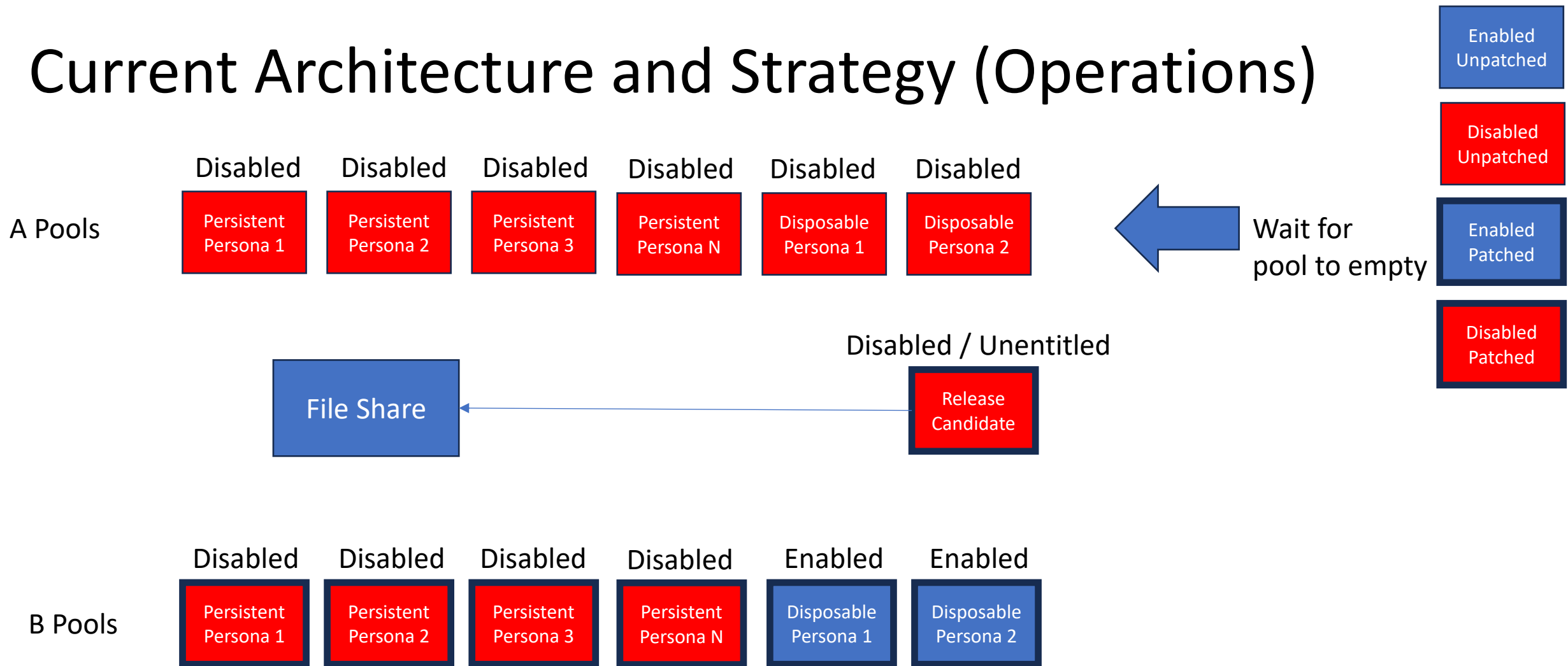
Current Architecture and Strategy (Operations)



Current Architecture and Strategy (Operations)

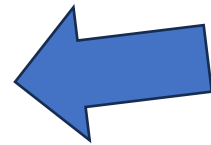
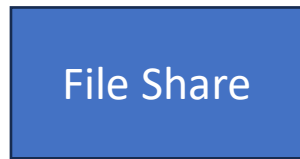
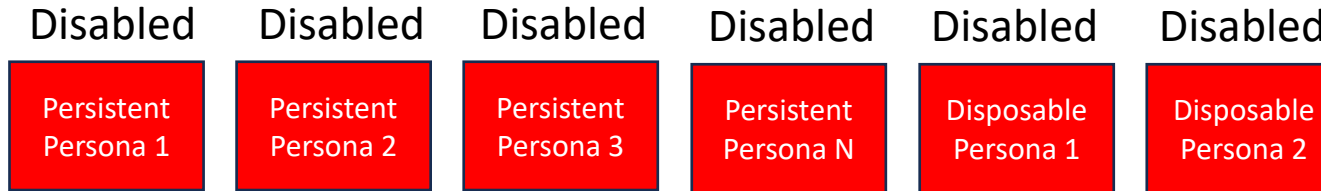


Current Architecture and Strategy (Operations)



Current Architecture and Strategy (Operations)

A Pools



Reboot File Server

Disabled / Unentitled



B Pools



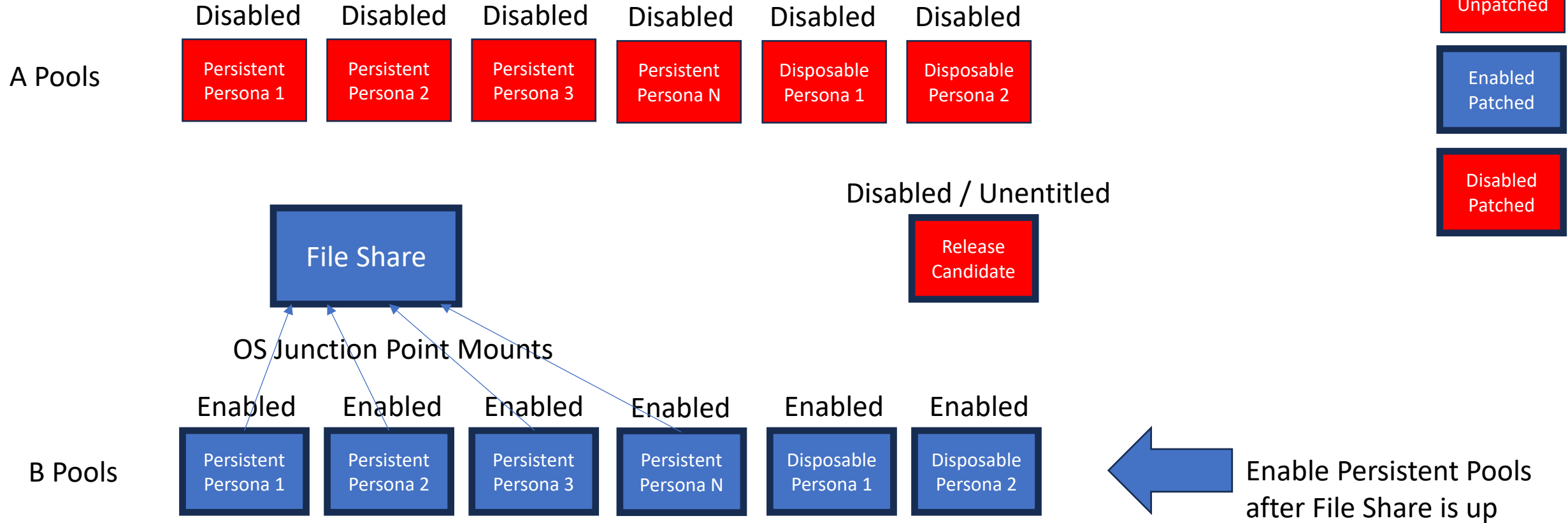
Enabled
Unpatched

Disabled
Unpatched

Enabled
Patched

Disabled
Patched

Current Architecture and Strategy (Operations)



Opportunities for Automation

- Challenge 1 – Spare Machines
 - Whatever the minimum desktop setting is for the disabled pools, those desktops will still be powered on
 - Need to Disable Provisioning on those pools and delete all VMs in the pool
- Challenge 2 – OS Upgrades
 - When upgrading the operating system you cannot simply point to the upgraded OS
 - Active Directory OU will stay the same and the new OS may have different GPO requirements
 - The new OS might have different TPM requirements (Windows 10 vs Windows 11)
 - Need to replace either the A or B side with the new OS and clone those pools

Opportunities for Automation – other tasks

- Challenge 3 – Send Message Functionality in Horizon
 - Often it is a good idea to notify connected users that maintenance is coming
 - The Horizon Administrator UI only lets you send messages to 100 selected users
 - Need to be able to send messages to all connected users
- Challenge 4 – Pool Backups
 - There isn't a built-in way to back up a pool's settings
 - Need to be able to back up all pool settings to JSON based configuration files
- Challenge 5 – Pushing an image to a pool

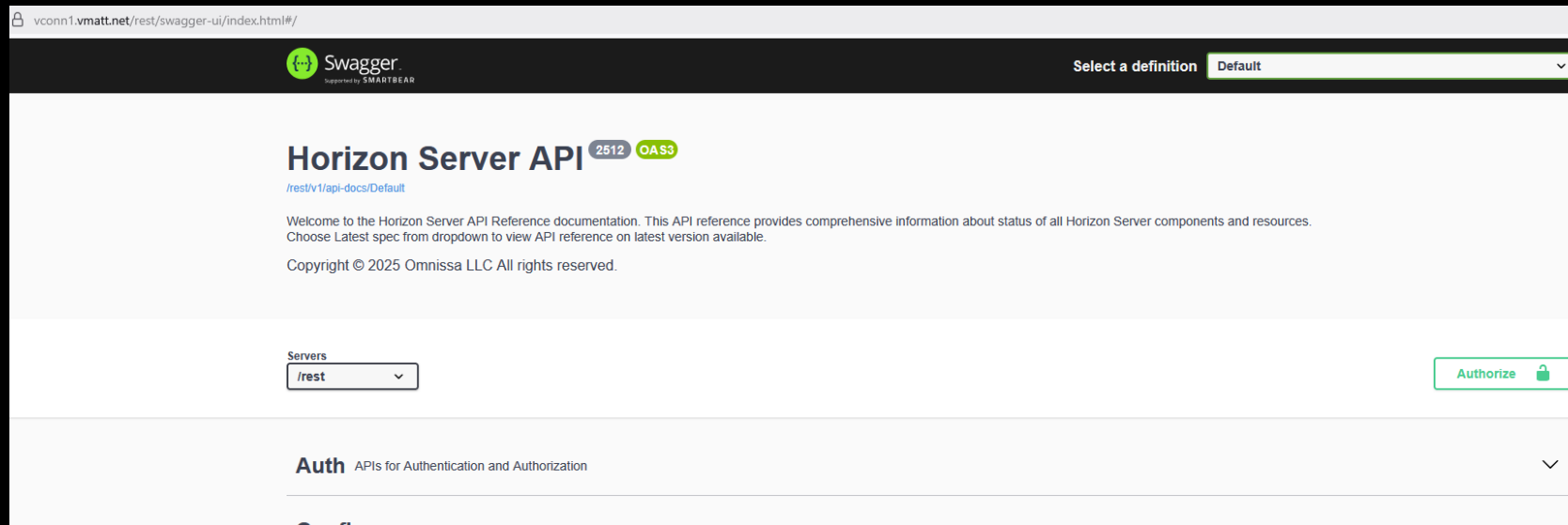


Horizon[®]

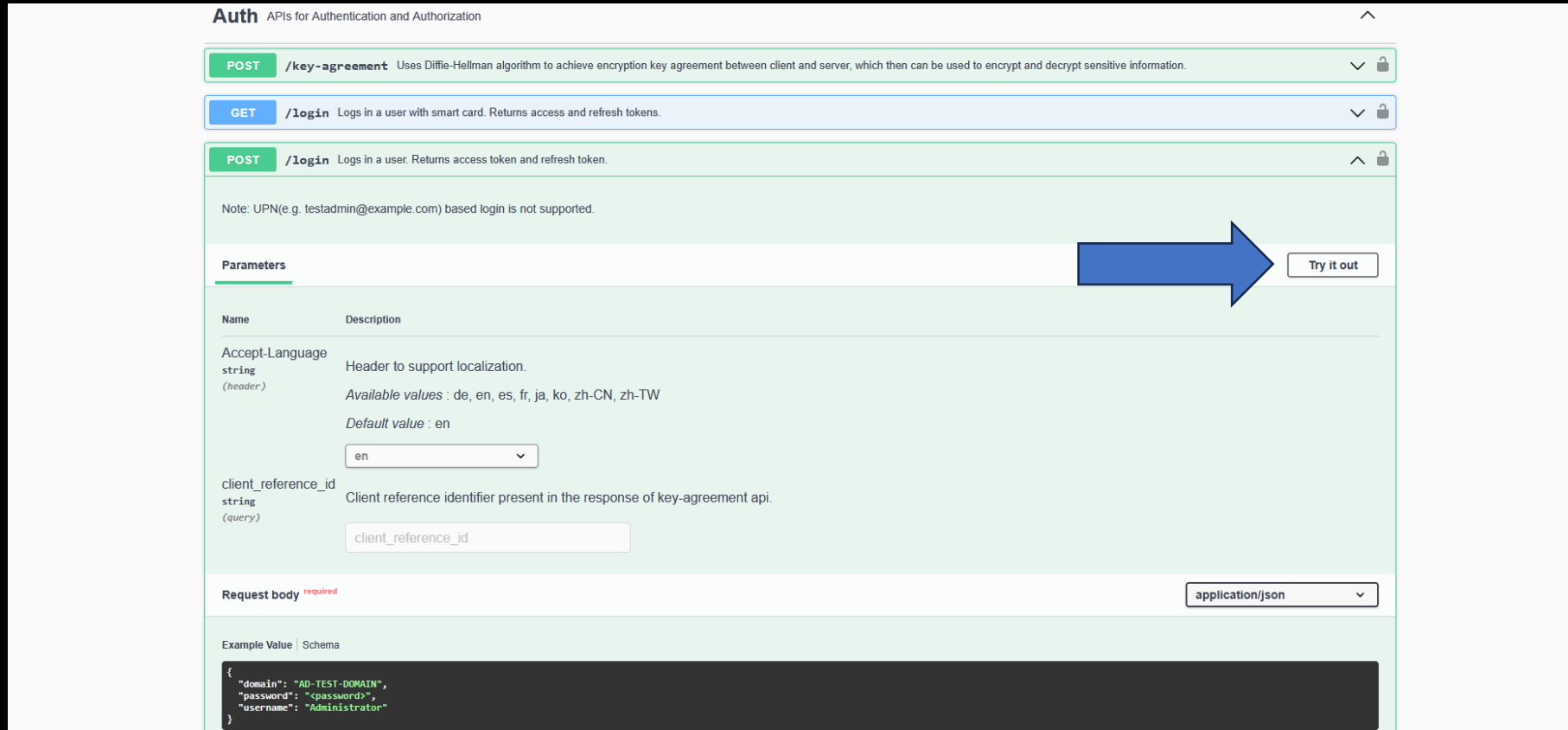
Using the REST API

Omnissa Horizon REST API

- Replaces the legacy Horizon SOAP API – Going away in Q2 2026
- Introduced in 2019 with Horizon 7.10
- Full Parity with SOAP API as of Horizon 8 Build 2312 (December 2023)
- Check it out at <https://yourConnSvrFQDN/rest/swagger-ui/index.html>



OmniSSA Horizon REST API – Using Swagger



Auth APIs for Authentication and Authorization

POST **/key-agreement** Uses Diffie-Hellman algorithm to achieve encryption key agreement between client and server, which then can be used to encrypt and decrypt sensitive information.

GET **/login** Logs in a user with smart card. Returns access and refresh tokens.

POST **/login** Logs in a user. Returns access token and refresh token.

Note: UPN(e.g. testadmin@example.com) based login is not supported.

Parameters

Name	Description
Accept-Language string (header)	Header to support localization. Available values : de, en, es, fr, ja, ko, zh-CN, zh-TW Default value : en en
client_reference_id string (query)	Client reference identifier present in the response of key-agreement api. client_reference_id

Request body required

Example Value | **Schema**

```
{
  "domain": "AD-TEST-DOMAIN",
  "password": "<password>",
  "username": "Administrator"
}
```

[Try it out](#)

OmniSSA Horizon REST API – Using Swagger

POST **/login** Logs in a user. Returns access token and refresh token.

⌵ 🔒

Note: UPN(e.g. testadmin@example.com) based login is not supported.

Parameters

Cancel Reset

Name	Description
Accept-Language string (header)	Header to support localization. en ▼ ☐ Send empty value
client_reference_id string (query)	Client reference identifier present in the response of key-agreement api. client_reference_id

Request body required

application/json ▼

```
{  "domain": "your-domain",  "password": "Your-P@ssw0rd",  "username": "Your-Login_id"}}
```

Execute

Clear

Omniassa Horizon REST API – Using Swagger

[illegible]

Grab access token (inside quotes)

Omnissa Horizon REST API – Using Swagger

Scroll to the top and click “authorize” – Sessions are typically 30 minutes

The image shows the Swagger UI for the Horizon Server API. At the top, there's a Swagger logo and a "Select a definition" dropdown set to "Default". Below this, the title "Horizon Server API" is displayed with version "2512" and "OAS3" tags. A link to the API docs is provided. A welcome message and copyright notice are also present. In the bottom left, there's a "Servers" dropdown set to "/rest". A large blue arrow points from this dropdown to the "Authorize" button, which is located in the bottom right of the main content area. Below the main content area, two smaller screenshots show the "Available authorizations" dialog. The left dialog shows the "Bearer (http, Bearer)" option selected, with a text input field containing a token and a blue arrow pointing to the "Authorize" button. The right dialog shows the same dialog after authorization, with the "Authorized" status and a "Logout" button, and a blue arrow pointing to the "Close" button.

Swagger
Supported by SMARTBEAR

Select a definition **Default**

Horizon Server API ²⁵¹² ^{OAS3}

</rest/v1/api-docs/Default>

Welcome to the Horizon Server API Reference documentation. This API reference provides comprehensive information about status of all Horizon Server components and resources.
Choose Latest spec from dropdown to view API reference on latest version available.

Copyright © 2025 Omnissa LLC All rights reserved.

Servers
/rest

Authorize

Available authorizations

Bearer (http, Bearer)

Value:
GdkjB5caMK4ReW8AkM0oA

Authorize Close

Available authorizations

Bearer (http, Bearer)

Authorized

Value: *****

Logout Close

OmniSSA Horizon REST API – Using Swagger

Try out a different endpoint –

Inventory APIs for inventory resources

GET `/inventory/v12/desktop-pools` Lists the desktop pools in the environment

This API supports **Pagination** and **Filters**.
For full information on using Filters, refer to 'Horizon Server REST Pagination and Filter Configuration'.
All the listed privileges are required -

Parameters Cancel

Name	Description
populate_additional_aws_fields boolean (query)	When true (default), populates AWS-specific fields. Set to false to improve performance when AWS data is not needed.
page integer (query)	page, if passed should be > 0.
size integer (query)	size, if passed should be > 0.
filter string (query)	filter expression built using fields with 'supported filters' as described in output model schema of this API.
sort_by string (query)	Field that would be used to sort the results.
order_by string (query)	Determines the ordering of results.

Execute Clear

Clear the values in the 1st, 4th, 5th
and 6th parameters and click
EXECUTE

Omnissa Horizon REST API – Using Swagger

Responses

Curl

[illegible]

Request URL

https://vconn1.vmath.net/rest/inventory/v12/desktop-pools?populate_additional_aws_fields=false&page=1&size=10

Server response

Code Details

200

Response body

```
{
  "id": "2140a164-1f31-4143-b6f7-c61619f2a9f5",
  "name": "WIN11-FC1",
  "display_name": "Windows 11 FC1",
  "access_group_id": "3ae27f8c-cc49-4273-b08d-b1467f9428bb",
  "enabled": true,
  "delete_in_progress": false,
  "enable_client_restrictions": false,
  "session_type": "DESKTOP",
  "cloud_managed": false,
  "cloud_assigned": false,
  "display_assigned_machine_name": false,
  "type": "MANUAL",
  "source": "VIRTUAL_CENTER",
  "vcenter_id": "be755822-7b58-46bc-a570-f12394e7fb6b",
  "user_assignment": "DEDICATED",
  "automatic_user_assignment": true,
  "allow_multiple_user_assignments": false,
  "transparent_page_sharing_scope": "VM",
  "session_settings": {
    "power_policy": "TAKE_NO_POWER_ACTION",
    "disconnected_session_timeout_policy": "NEVER",
    "allow_users_to_reset_machines": true,
    "allow_multiple_sessions_per_user": false
  },
  "display_protocol_settings": {
    "display_protocols": [

```

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
content-security-policy: default-src 'none';connect-src 'self' https;font-src 'self' data;img-src 'self' data;base-uri 'none';form-action 'none';frame-ancestors 'none';script-src 'self';style-src 'self' 'unsafe-inline'
content-type: application/json
expires: 0
pragma: no-cache
referrer-policy: strict-origin-when-cross-origin
strict-transport-security: max-age=31536000 ; includeSubDomains
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-permitted-cross-domain-policies: none
x-robots-tag: none
x-total-count: 5
x-xss-protection: 0
```

Omnissa Horizon REST API – Using Swagger

[illegible]

Using Powershell



Basic Functions for your scripts

- Get-TokenExpiry

- Looks to see if \$token exists
 - If it does, it gets the expiration time
 - If not, it skips the check
- Extracts the expiration time from token
- Returns an object \$expirationTime
- Value is in UTC

```
# Function to check for a valid Horizon REST API token
14 references
function Get-TokenExpiry {

    # If token is null, empty, or whitespace – skip the check entirely
    if ([string]::IsNullOrEmpty($token)) {
        Write-Host "No token found – skipping expiration check." -ForegroundColor Yellow
        return $null
    }

    try {
        $tokenParts = $token.Split('.')

        $decodedPayload = [System.Text.Encoding]::UTF8.GetString(
            [Convert]::FromBase64String($tokenParts[1])
        )

        $decodedPayloadObj = $decodedPayload | ConvertFrom-Json
        $expirationTimestamp = $decodedPayloadObj.exp

        $expirationTime = [DateTimeOffset]::FromUnixTimeSeconds($expirationTimestamp).UtcDateTime

        Write-Host "Token expires at: $expirationTime"
        return $expirationTime
    }
    catch {
        Write-Host "Token is invalid or unreadable – skipping expiration check." -ForegroundColor Yellow
        return $null
    }
}
```

Basic Functions for your scripts

• Get-HorizonToken

- Prompts for credentials
- Executes a POST to the /rest/login API endpoint
- Returns an object \$token
- Requires \$connectionServer to be valid in your script

Important Security Note

Source: PowerShell Security Best Practices (Microsoft Learn)

<https://learn.microsoft.com/powershell/scripting/security/secure-strings>

Microsoft explicitly states:

“Use Get-Credential to prompt the user for credentials. This cmdlet returns a PSCredential object that contains a SecureString password.”

And:

“If you must use a plaintext password—for example, to call an API—convert the SecureString only at the point of use.”

```
function Get-HorizonToken {  
  
    Write-Host "Token invalid - Re-authenticating" -ForegroundColor Yellow  
  
    # Prompt for Horizon admin credentials  
    $creds = Get-Credential -Message "Enter Horizon admin credentials"  
  
    # Extract username + password  
    $username = $creds.UserName  
    $password = $creds.GetNetworkCredential().Password  
  
    # If user typed DOMAIN\username, split it  
    if ($username -match "\\") {  
        $domain, $username = $username -split "\\", 2  
    }  
  
    # Build authentication URI  
    $authUri = "$connectionServer/rest/login"  
  
    # Build JSON body  
    $body = @{  
        username = $username  
        password = $password  
        domain   = $domain  
    } | ConvertTo-Json  
  
    # Send authentication request  
    $response = Invoke-RestMethod -Method Post -Uri $authUri ` -Body $body ` -ContentType "application/json" ` -SkipCertificateCheck  
  
    # Extract token  
    $token = $response.access_token  
  
    Write-Host "Authenticating to " $authUri  
  
    return $token  
}
```

Basic Functions for your scripts

Combine the two functions in the body of your scripts

```
Write-Host "Current UTC Time:" (Get-Date).ToUniversalTime().ToString("MM/dd/yyyy HH:mm:ss")
$tokenExpirationTime = Get-TokenExpiry

if ($tokenExpirationTime -gt (Get-Date -AsUTC)) {
    $tokenStatus = $true
} else {
    $token = Get-HorizonToken
    $tokenExpirationTime = Get-TokenExpiry
}
```

Basic Functions for your scripts

- Get-HorizonVersion

- Hits the Horizon REST API documentation at /rest/v1/api-docs/Default
- Searches
- Returns an object \$version
- Helps when pointing your server URI to the proper endpoint
- \$horizonVersion = Get-HorizonVersion
- \$connectionServer needs to exist

Raw Data within api-docs

```
Function Get-HorizonVersion {  
  
    $apiDocsUri = "$connectionServer/rest/v1/api-docs/Default"  
  
    $headers = @{  
        Authorization = "Bearer $token"  
    }  
  
    $apiDocs = Invoke-RestMethod -Method GET -Uri $apiDocsUri -Headers $headers  
    $version = $apiDocs.info.version  
    return $version  
}
```

```
{  
    "openapi": "3.0.1",  
    "info": {  
        "description": "Welcome to the Horizon Server API Reference  
documentation. This API reference provides comprehensive  
information about status of all Horizon Server components and  
resources. \u003Cbr\u003E \nChoose Latest spec from dropdown to  
view API reference on latest version available.",  
        "license": {  
            "name": "Copyright \u00a9 2025 Omnisia LLC All rights reserved."  
        },  
        "title": "Horizon Server API",  
        "version": "2512"  
    },  
}
```

Basic Functions for your scripts

- Using Get-HorizonVersion

- Validate token
- Get Horizon Version
- Desktop Pools endpoint
 - Use switch command to establish newest desktop pools API endpoint version
 - Needs to be updated manually
 - Get all pools example below

```
# Get all pools
$poolUri = "$connectionServer/rest/inventory/$dpVer/desktop-pools"
$pool = Invoke-RestMethod -Method GET -Uri $poolUri -Headers @{
    "Authorization" = "Bearer $token"
    "Accept"        = "application/json"
} # -SkipCertificateCheck
```

```
Write-Host "Current UTC Time:" (Get-Date).ToUniversalTime().ToString("MM/dd/yyyy HH:mm:ss")
$tokenExpirationTime = Get-TokenExpiry

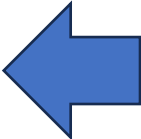
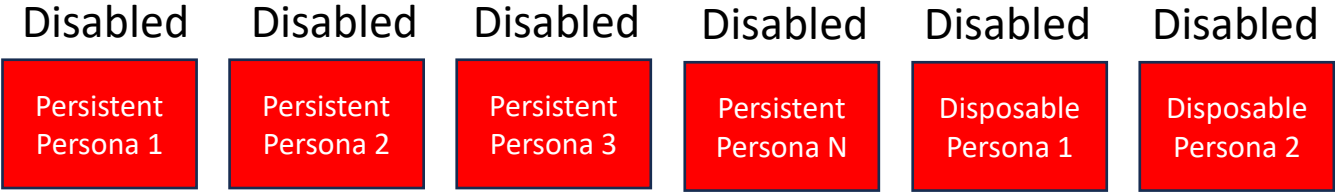
$horizonVersion = Get-HorizonVersion
Write-host $horizonVersion

if ($tokenExpirationTime -gt (Get-Date -AsUTC)) {
    $tokenStatus = $true
} else {
    $token = Get-HorizonToken
    $tokenExpirationTime = Get-TokenExpiry
}

# Desktop Pools - Key API Endpoint Versions
switch ($horizonVersion) {
    "2512" { $dpVer = "v12" }
    "2506" { $dpVer = "v11" }
    "2503" { $dpVer = "v10" }
    "2412" { $dpVer = "v10" }
    "2406" { $dpVer = "v8" }
    "2312" { $dpVer = "v8" }
    "2309" { $dpVer = "v7" }
    default { $dpVer = "v12" }
}
```

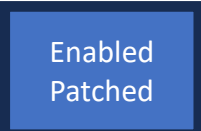
Back to our Architecture

A Pools

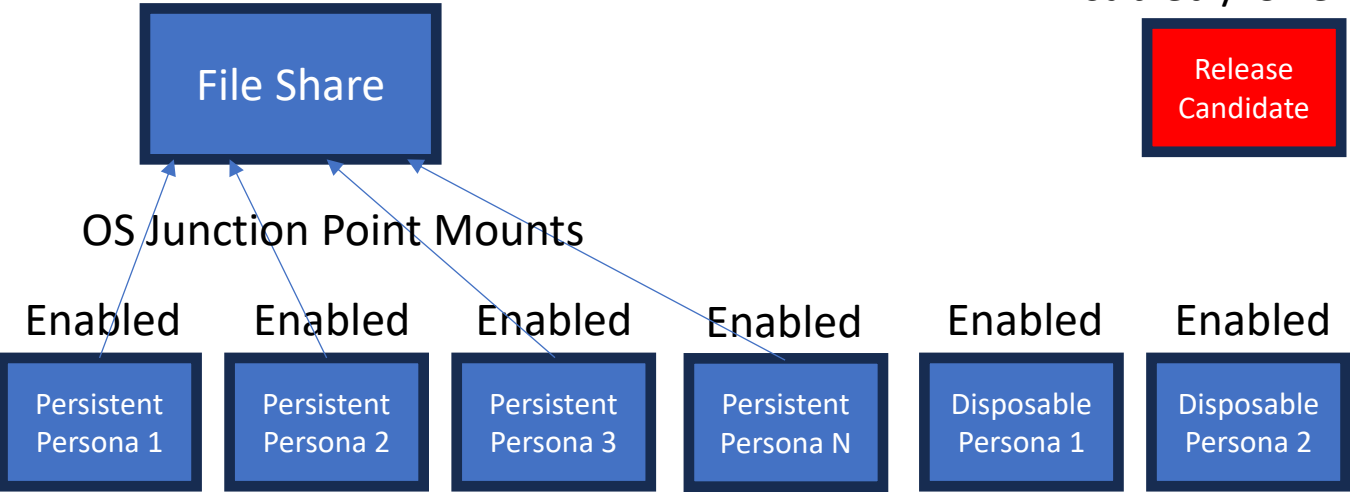


These VDI Pools have unusable spare machines taking up infra resources

Disabled / Unentitled



B Pools



Opportunities for Automation

- Challenge 1 – Spare Machines

- Whatever the minimum desktop setting is for the disabled pools, those desktops will still be powered on
- Need to Disable Provisioning on those pools and delete all VMs in the pool

Let's fix it!



Opportunities for Automation – Remove Spares

- (Step 1) Modify Pool Settings
 - Set “poolNameToModify” at top of script along with connection server and domain
 - Get all pools
 - Find the pool ID associated with \$poolNameToModify
 - Extract all pool settings for that specific pool to an object \$updated
 - Ask the user if they would like to disable provisioning and/or the pool itself
 - Extract the values of the answers and make a Boolean
 - Change the values to \$updated.enable_provisioning and \$updated.enabled
 - Convert \$updated to a JSON
 - PUT the JSON to “/rest/inventory/\$dpver/desktop-pools/\$poolId”

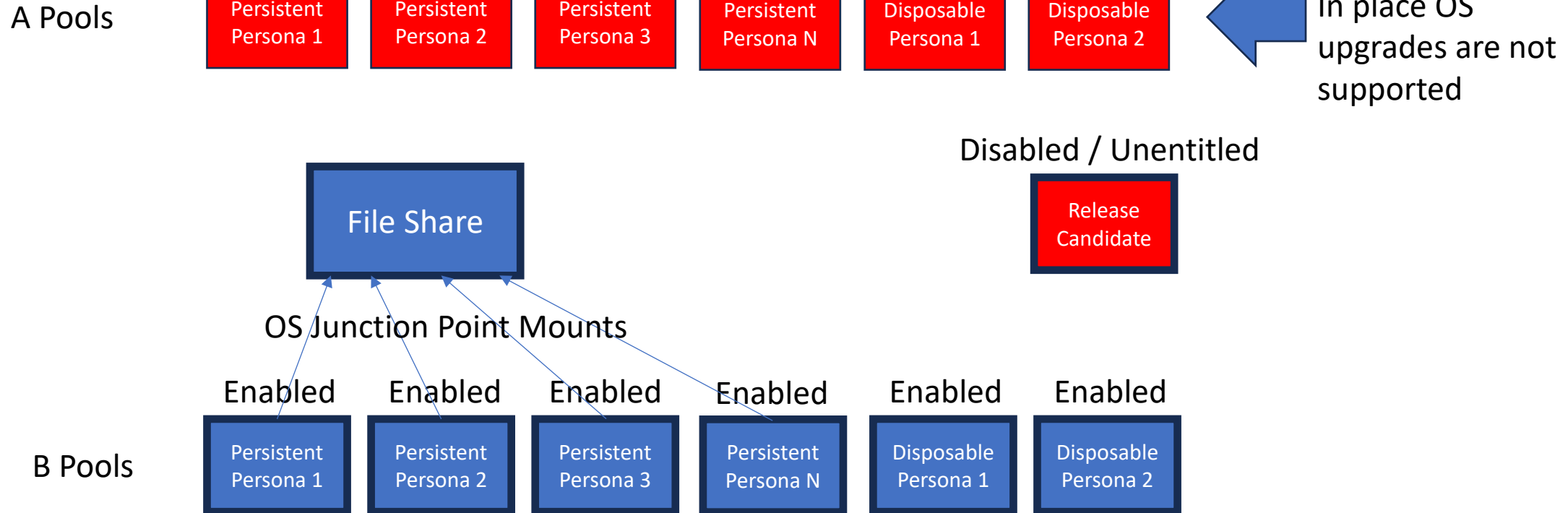
Opportunities for Automation – Remove Spares

- (Step 2) Delete all machines in a pool
 - Get a list of all machines
 - Find machines where the pool ID matches and the state is AVAILABLE
 - If machines > 0
 - Build a list of machine ID's
 - Form JSON to delete from disk and not force logoff
 - If machines not > 0
 - Inform the user there are no machines found
- NOTE: If you left provisioning on in Step 1, machines will be recreated

Opportunities for Automation – Remove Spares

DEMO

What about Operating System Upgrades?



Enabled
Unpatched

Disabled
Unpatched

Enabled
Patched

Disabled
Patched

Opportunities for Automation

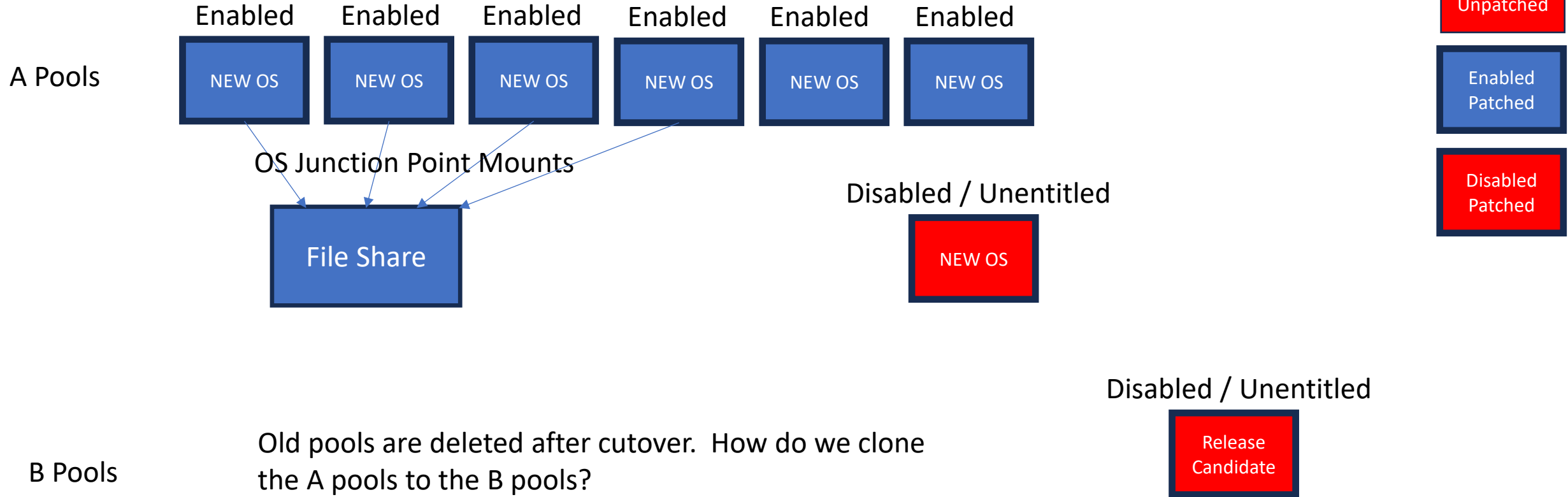
- Challenge 2 – OS Upgrades

- When upgrading the operating system you cannot always point to the upgraded OS
 - Active Directory OU will stay the same and the new OS may have different GPO requirements
 - The new OS might have different TPM requirements (Windows 10 vs Windows 11)
- Need to replace either the A or B side with the new OS and clone those pools

Let's fix it!



What about Operating System Updates?



Opportunities for Automation

- Challenge 2 – Clone Desktop Pool to a new Pool - TO DO LIST
 - Run functions to Validate / Create API Token, determine Horizon version
 - Get all pool information
 - Match pool name to pool ID and import that pool to a PS object \$currentPool
 - Remove the “id” object so it gets recreated
 - Change the pool “name” and “naming pattern”
 - Ask user if they want to enable the pool and/or enable provisioning
 - Import changes and clone the pool with new properties
 - Ask user if they want to clone entitlements from original pool

Opportunities for Automation

- Challenge 2 – Clone Desktop Pool to a new Pool

DEMO

Opportunities for Automation

- Challenge 3 – Send Message Functionality in Horizon
 - Often it is a good idea to notify connected users that maintenance is coming
 - The Horizon Administrator UI only lets you send messages to 100 selected users
 - Need to be able to send messages to all connected users

Let's fix it!



🔄 Updated 04/03/2026, 9:40 AM ▾

Sessions 4 ▾

Problem vCenter VMs 0 ▾

Problem Capacity Provider ... 0 ▾

Problem RDS Hosts 0 ▾

Events 2 ▾

System Health 3 ▾

Monitor ▾

Dashboard

Events

Infrastructure

Sessions

Help Desk

Users and Groups

Sessions

Sessions All ▾

DISCONNECT SESSION

LOGOFF SESSION

RESTART DESKTOP

RESET VIRTUAL MACHINE

SEND MESSAGE



☐	User	Type	Pool or Farm	Bypass Session Timeout	DNS Name	Client ID
☐	vmatt.net\uo5221ps	Desktop	Windows Pool 1		win1a04.vmatt.net	cf50e77c2cd2d7d9c5
☐	vmatt.net\uo5221ps	Desktop	Windows Pool 1		win1b01.vmatt.net	cf50e77c2cd2d7d9c5
☐	vmatt.net\uo5221ps	Desktop	Windows Pool 2		win2b08.vmatt.net	cf50e77c2cd2d7d9c5
☐	vmatt.net\uo5221ps	Desktop	Windows Pool 2		win2a04.vmatt.net	cf50e77c2cd2d7d9c5
MANAGE COLUMNS						

Rows per page 20 ▾

1 - 4 of 4 rows

Opportunities for Automation

- Challenge 3 – Send Message Functionality in Horizon
 - Run functions to Validate / Create API Token
 - Get a list of all sessions and create an array \$sessionIds
 - Ask user for the message to send, message type, and verification to send
 - Create \$messagePayload JSON containing the message info and the array of sessions
 - POST JSON body to /rest/inventory/v1/sessions/action/send-message

Opportunities for Automation

- Challenge 4 – Pool Backups
 - There isn't a built-in way to back up a pool's settings
 - Need to be able to back up all pool settings to JSON based configuration files

Let's fix it!



Opportunities for Automation

- Challenge 4 – Pool Backups – TO DO
 - Establish Connection Server and Domain objects
 - Validate / create token / get horizon build version and assign \$dpVer
 - Establish export path object
 - If path doesn't exist, create the folder
 - GET all pools using /rest/inventory/\$dpVer/desktop-pools endpoint
 - Iterate through all pools
 - Get pool ID
 - Save Pool settings to export folder using the pool name associated with the pool ID

Opportunities for Automation

- Challenge 5 – Pushing an image to a pool
 - Push Image is a manual process
 - Need to create a script that
 - Allows for a change of compute resources
 - Allows for a change of snapshot
 - Allows for a change of gold image VM name

Let's fix it!





Link to my GitHub Repo



Link to this
Presentation



Thank You!

Your Feedback Is Important

Please take a minute to complete the survey session in the mobile app.

1. Navigate to this session
2. Take this survey
3. Enjoy your next session!

Download the VMUG Connect app

Make the most of your time onsite.

Google Store



App Store



From Connect to Community -

What's Next?

Make the Most of Your Time at Connect!

- Take a **Certification** Exam – Prove your skills and boost your career.
- Meet Your **Local Leaders** – Find out how to stay involved year-round.
- Explore **Partner Solutions** – Get hands-on with the latest VMware innovations.
- Upgrade to **VMUG Advantage** – Unlock exclusive training, personal-use homelab licenses, and more!

Keep Exploring, Learning, and Networking!

1. Join your local VMUG Chapter
2. Tap Into Advantage Benefits
3. Follow VMUG on Social – Stay in the loop and share your experience! #VMUGConnect
4. Mark Your Calendar for VMware Explore – Be part of the biggest VMware event of the year. VMUG cannot wait to see you there
5. Look out for Global Virtual Events on vmug.com



Ready to take the next step?
Visit [VMUG.com](https://vmug.com) and stay connected!